



Informatik II – Machine Learning

Informatik II – Woche 2

19. / 20. Mai 2026

Heutiges Programm

Logistische Regression

Neural Networks

Programmierübung

Training

Programmierübung

Cross-Validation

1. Logistische Regression

Warum logistische Regression?

Lineare Regression	Logistische Regression
sagt ein kontinuierliches y voraus	sagt eine Klassenwahrscheinlichkeit voraus
z. B. Preis, Temperatur, Punktzahl	z. B. Spam? Betrug? Krebs?
Ausgabe: eine beliebige reelle Zahl	Ausgabe: eine Zahl in $[0, 1]$

Warum logistische Regression?

Lineare Regression	Logistische Regression
sagt ein kontinuierliches y voraus	sagt eine Klassenwahrscheinlichkeit voraus
z. B. Preis, Temperatur, Punktzahl	z. B. Spam? Betrug? Krebs?
Ausgabe: eine beliebige reelle Zahl	Ausgabe: eine Zahl in $[0, 1]$

Gleiche Idee (ein lineares Modell), andere Frage (eine Ja/Nein-Antwort).

Der Datensatz

Beispiel. Vorhersagen, ob eine E-Mail Spam ist.

x_i (Merkmale)		y_i (Label)	
"\$\$\$ free!"	Links		
1	4	1	(Spam)
0	1	0	(kein Spam)
0	0	0	(kein Spam)

Der Datensatz

Beispiel. Vorhersagen, ob eine E-Mail Spam ist.

$\mathbf{x}_i$ (Merkmale)		y_i (Label)	
"\$\$\$ free!"	Links		
1	4	1	(Spam)
0	1	0	(kein Spam)
0	0	0	(kein Spam)

Gegeben sind n Datenpunkte. Jeder ist ein Merkmalsvektor $\mathbf{x}_i \in \mathbb{R}^D$ zusammen mit einem binären Label $y_i \in \{0, 1\}$.

Der Datensatz

Beispiel. Vorhersagen, ob eine E-Mail Spam ist.

$\mathbf{x}_i$ (Merkmale)		y_i (Label)	
"\$\$\$ free!"	Links		
1	4	1	(Spam)
0	1	0	(kein Spam)
0	0	0	(kein Spam)

Gegeben sind n Datenpunkte. Jeder ist ein Merkmalsvektor $\mathbf{x}_i \in \mathbb{R}^D$ zusammen mit einem binären Label $y_i \in \{0, 1\}$. **Ziel.** Eine Funktion finden, die die Wahrscheinlichkeit $\hat{y}_i \in [0, 1]$ vorhersagt, dass das Label 1 ist.

Von einer Zahl zu einer Wahrscheinlichkeit

Schritt 1. Linearen Score berechnen, wie bei der linearen Regression:

$$z = \mathbf{w} \cdot \mathbf{x}$$

- Problem: z kann *jede* reelle Zahl sein, eine Wahrscheinlichkeit muss aber in $[0, 1]$ liegen.

Von einer Zahl zu einer Wahrscheinlichkeit

Schritt 1. Linearen Score berechnen, wie bei der linearen Regression:

$$z = \mathbf{w} \cdot \mathbf{x}$$

→ Problem: z kann *jede* reelle Zahl sein, eine Wahrscheinlichkeit muss aber in $[0, 1]$ liegen.

Schritt 2. z durch die **Sigmoidfunktion** σ quetschen, um in $[0, 1]$ zu landen:

$$\hat{y} = \sigma(z) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

Von einer Zahl zu einer Wahrscheinlichkeit

Schritt 1. Linearen Score berechnen, wie bei der linearen Regression:

$$z = \mathbf{w} \cdot \mathbf{x}$$

→ Problem: z kann *jede* reelle Zahl sein, eine Wahrscheinlichkeit muss aber in $[0, 1]$ liegen.

Schritt 2. z durch die **Sigmoidfunktion** σ quetschen, um in $[0, 1]$ zu landen:

$$\hat{y} = \sigma(z) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

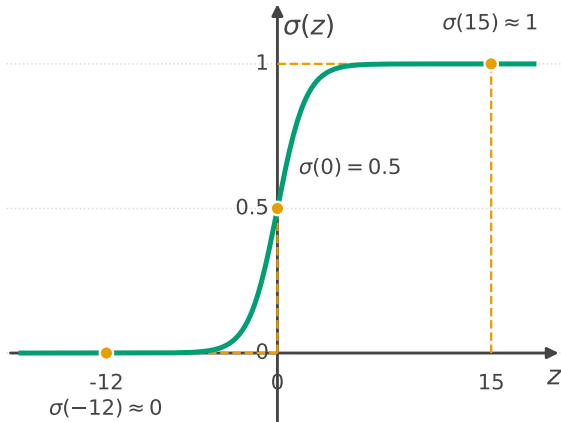
Logistische Regression = lineares Modell +
Sigmoid-Quetsche.

Die Sigmoidfunktion

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

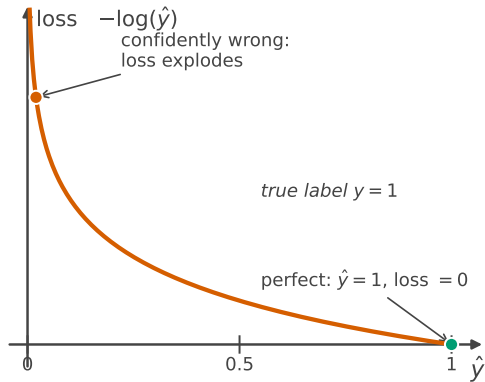
- grosses positives $z \rightarrow \hat{y} \approx 1$
- $z = 0 \rightarrow \hat{y} = 0.5$
- grosses negatives $z \rightarrow \hat{y} \approx 0$

Gross positiv: sicheres Ja.
Gross negativ: sicheres Nein.
Um 0 herum: unsicher.



Kreuzentropie-Verlust

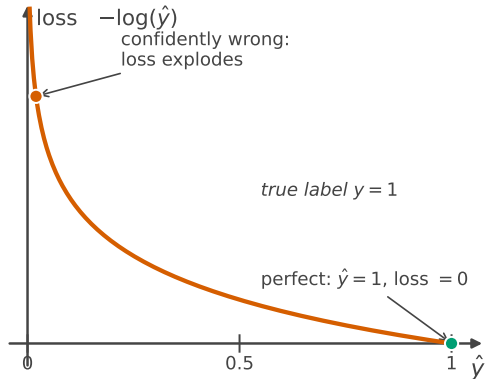
Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.



Kreuzentropie-Verlust

Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.

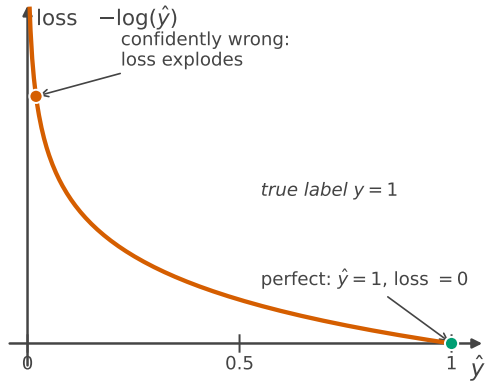
■ $\hat{y} = 1 \rightarrow \text{Verlust} = 0$.



Kreuzentropie-Verlust

Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.

- $\hat{y} = 1 \rightarrow \text{Verlust} = 0$.
- $\hat{y} \rightarrow 0 \rightarrow \text{Verlust} \rightarrow \infty$.



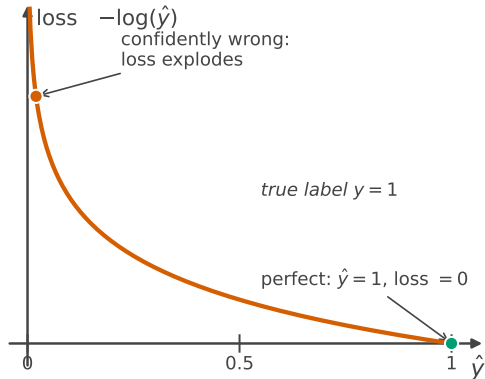
Kreuzentropie-Verlust

Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.

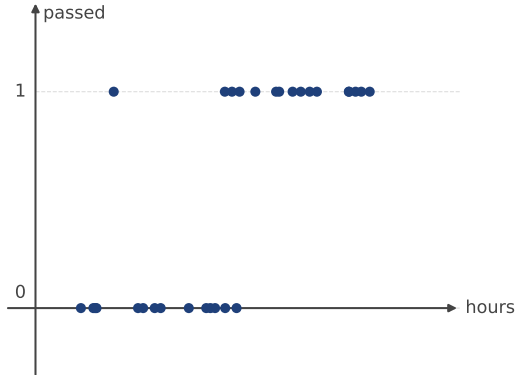
- $\hat{y} = 1 \rightarrow \text{Verlust} = 0.$
- $\hat{y} \rightarrow 0 \rightarrow \text{Verlust} \rightarrow \infty.$

Gemittelt über den Datensatz:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

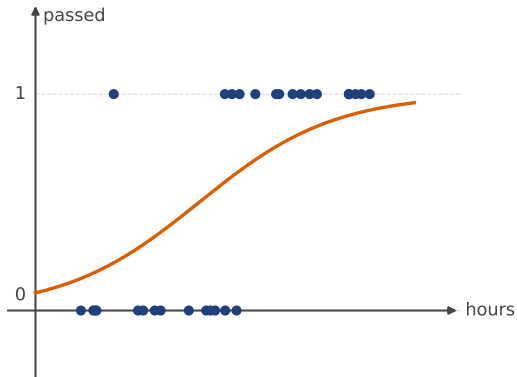


Ein Beispiel



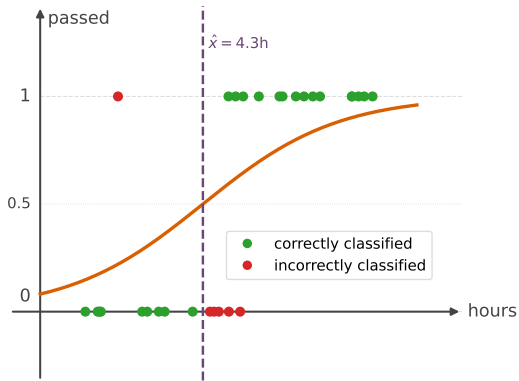
- $n = 30$ Studierende. Merkmal: gelernte Stunden. Label: bestanden ($y = 1$) oder nicht bestanden ($y = 0$).

Ein Beispiel



- $n = 30$ Studierende. Merkmal: gelernte Stunden. Label: bestanden ($y = 1$) oder nicht bestanden ($y = 0$).
- Das logistische Regressionsmodell anpassen.

Ein Beispiel



- $n = 30$ Studierende. Merkmal: gelernte Stunden. Label: bestanden ($y = 1$) oder nicht bestanden ($y = 0$).
- Das logistische Regressionsmodell anpassen.
- **Entscheidungsgrenze**: dort, wo $\hat{y} = 0.5$ gilt. Rechts davon: Vorhersage "bestanden". Links: Vorhersage "nicht bestanden".

Zusammenfassung

Logistische Regression = **lineares Modell** + **Sigmoid-Quetsche**.

Sie liefert eine **Wahrscheinlichkeit** $\hat{y} \in [0, 1]$.

Die **Entscheidungsgrenze** liegt dort, wo $\hat{y} = 0.5$ ist.

Zusammenfassung

Logistische Regression = **lineares Modell** + **Sigmoid-Quetsche**.

Sie liefert eine **Wahrscheinlichkeit** $\hat{y} \in [0, 1]$.

Die **Entscheidungsgrenze** liegt dort, wo $\hat{y} = 0.5$ ist.

- Eingesetzt für binäre Ausgänge: ja/nein, Spam/kein Spam, krank/gesund.
- Trainiert wird sie durch Minimierung des Kreuzentropie-Verlusts.

2. Theoretische Übungen

Übung 1: Linear oder logistisch?

Für jede der folgenden Vorhersageaufgaben entscheiden, ob **lineare** oder **logistische** Regression einzusetzen ist, und kurz begründen, warum.

- (a) Die morgige Temperatur in Zürich vorhersagen.
- (b) Vorhersagen, ob ein Patient Diabetes hat (ja / nein).
- (c) Den Preis eines Hauses anhand seiner Grösse und Lage vorhersagen.
- (d) Vorhersagen, ob eine Banktransaktion betrügerisch ist.
- (e) Die Abschlussnote (0–100) einer studierenden Person vorhersagen.

Übung 1: Linear oder logistisch?

Für jede der folgenden Vorhersageaufgaben entscheiden, ob **lineare** oder **logistische** Regression einzusetzen ist, und kurz begründen, warum.

- (a) Die morgige Temperatur in Zürich vorhersagen.
- (b) Vorhersagen, ob ein Patient Diabetes hat (ja / nein).
- (c) Den Preis eines Hauses anhand seiner Grösse und Lage vorhersagen.
- (d) Vorhersagen, ob eine Banktransaktion betrügerisch ist.
- (e) Die Abschlussnote (0–100) einer studierenden Person vorhersagen.

- ❗ (a) **Linear**, denn die Temperatur ist ein kontinuierlicher Wert.
- (b) **Logistisch**, denn der Ausgang ist binär (ja / nein).
- (c) **Linear**, denn der Preis ist ein kontinuierlicher Wert.
- (d) **Logistisch**, denn der Ausgang ist binär (Betrug / kein Betrug).
- (e) **Linear**, denn die Note ist ein kontinuierlicher Wert.

Übung 2: Das schlechte Modell finden

Zwei logistische Regressionsmodelle werden auf demselben Datensatz von 20 Patienten trainiert (Label: krank, ja/nein).

Modell 1 auf den *Trainingsdaten*: 0.999, 0.001, 0.998, 0.002, 0.997, 0.001, ...

Modell 2 auf den *Trainingsdaten*: 0.51, 0.49, 0.53, 0.48, 0.50, 0.52, ...

- (a) Was “sagt” jedes Modell mit seinen Ausgaben?
- (b) Welchem Modell ist auf neuen Patienten eher zu trauen, und warum?

Übung 2: Das schlechte Modell finden

Zwei logistische Regressionsmodelle werden auf demselben Datensatz von 20 Patienten trainiert (Label: krank, ja/nein).

Modell 1 auf den *Trainingsdaten*: 0.999, 0.001, 0.998, 0.002, 0.997, 0.001, ...

Modell 2 auf den *Trainingsdaten*: 0.51, 0.49, 0.53, 0.48, 0.50, 0.52, ...

- (a) Was “sagt” jedes Modell mit seinen Ausgaben?
 - (b) Welchem Modell ist auf neuen Patienten eher zu trauen, und warum?
- ❗ (a) Modell 1 ist auf jedem Trainingsbeispiel fast völlig sicher. Modell 2 ist kaum besser als ein Münzwurf.
- (b) Keines ist ideal, doch **Modell 1 ist der gefährlichere Fall**: Es hat höchstwahrscheinlich *überangepasst*, also die 20 Beispiele auswendig gelernt, statt ein allgemeines Muster zu erfassen. Auf neuen Patienten ist seine Sicherheit unzuverlässig. Modell 2 hat *unterangepasst*: Es hat kaum etwas gelernt.

Über- und Unteranpassung folgen formal in der Trainings-Lektion.

Übung 3: Die Sigmoidfunktion lesen

Ohne zu rechnen, die folgenden Fragen zur Sigmoidfunktion $\sigma(z) = \frac{1}{1+e^{-z}}$ beantworten.

- (a) Ein Modell berechnet $\mathbf{w} \cdot \mathbf{x} = 15$. Was ist $\hat{y}$ ungefähr? Welche Klasse sagt das Modell voraus?
- (b) Ein Modell berechnet $\mathbf{w} \cdot \mathbf{x} = -12$. Was ist $\hat{y}$ ungefähr?
- (c) Ein Modell berechnet $\mathbf{w} \cdot \mathbf{x} = 0$. Was ist $\hat{y}$ exakt?
- (d) Beobachtet wird $\hat{y} = 0.5$ für jeden Eingabewert im Datensatz. Was sagt das über das Modell aus?

Übung 3: Die Sigmoidfunktion lesen (Lösung)

- ❗ (a) $\hat{y} \approx 1$, das Modell sagt also Klasse 1 mit hoher Sicherheit voraus.
- (b) $\hat{y} \approx 0$, das Modell sagt also Klasse 0 mit hoher Sicherheit voraus.
- (c) $\hat{y} = 0.5$ exakt, das Modell ist also maximal unsicher.
- (d) Das Modell hat nichts gelernt: $\mathbf{w} \cdot \mathbf{x} = 0$ für jede Eingabe, d. h. alle Gewichte sind null (oder die Merkmale haben keinen Einfluss auf die Ausgabe).

3. Neural Networks

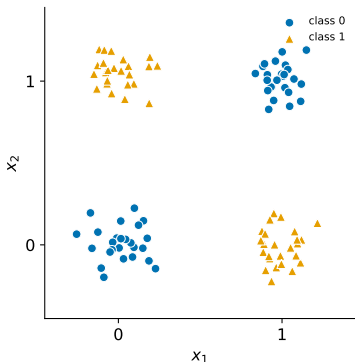
Das Problem: Lineare Modelle sind nicht genug

Lineare Regression und logistische Regression können nur **lineare** Entscheidungsgrenzen lernen.

Das Problem: Lineare Modelle sind nicht genug

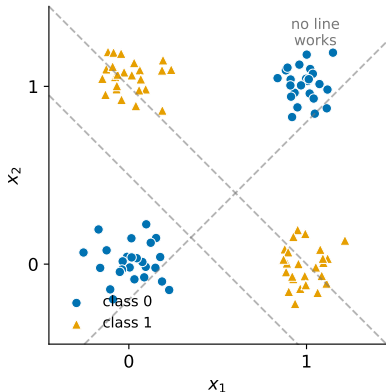
Lineare Regression und logistische Regression können nur **lineare** Entscheidungsgrenzen lernen.

Das XOR-Problem: Wie lässt sich die Entscheidungsgrenze für den folgenden Datensatz vorhersagen?



Das Problem und die Lösung: XOR

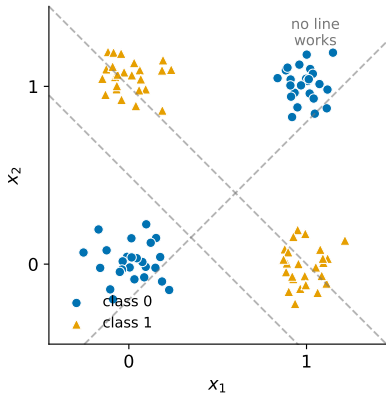
Lineares Modell (scheitert)



Lineare Regression kann
XOR-Daten nicht mit einer
einzigen Geraden trennen.

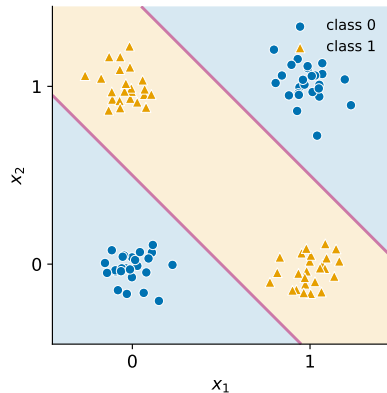
Das Problem und die Lösung: XOR

Lineares Modell (scheitert)



Lineare Regression kann XOR-Daten nicht mit einer einzigen Geraden trennen.

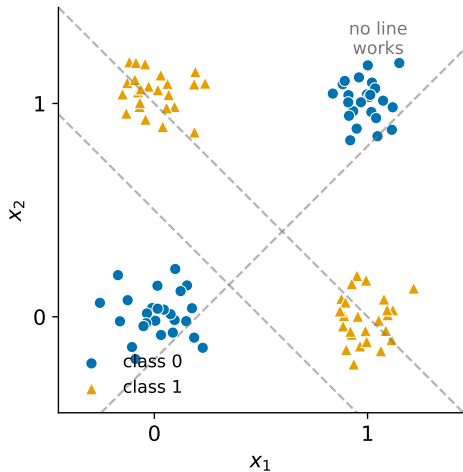
Neuronales Netz (funktioniert!)



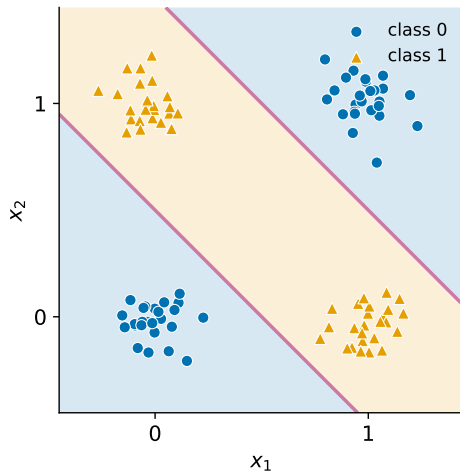
Ein zweischichtiges Netz mit Aktivierungsfunktionen lernt die richtige Grenze.

Das Problem und die Lösung: XOR

Lineares Modell (scheitert)



Neuronales Netz (funktioniert!)



Kernidee: Versteckte Schichten transformieren die Daten so, dass die

Neural Network Architektur: Schichten und Gewichte

Ein neuronales Netz ist ein Stapel von **Schichten**. Jede Schicht hat:

Neural Network Architektur: Schichten und Gewichte

Ein neuronales Netz ist ein Stapel von **Schichten**. Jede Schicht hat:

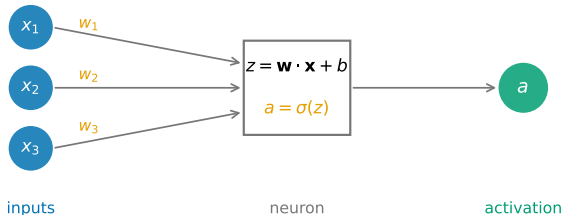
- **Eingaben**: Merkmalsvektor $\mathbf{x}$ (oder Ausgabe der vorigen Schicht)
- **Gewichtsmatrix** $\mathbf{W}$: gelernte Parameter, Eingaben $\rightarrow$ Neuronen
- **Bias-Vektor** $\mathbf{b}$: gelernte Verschiebung
- **Gewichtete Summe**: $\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$
- **Aktivierungsfunktion** $\sigma(\cdot)$: nicht-lineare Transformation auf $\mathbf{z}$

Neural Network Architektur: Schichten und Gewichte

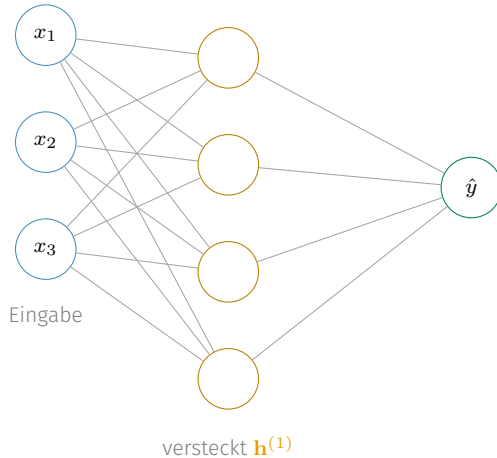
Ein neuronales Netz ist ein Stapel von **Schichten**. Jede Schicht hat:

- **Eingaben**: Merkmalsvektor $\mathbf{x}$ (oder Ausgabe der vorigen Schicht)
- **Gewichtsmatrix** $\mathbf{W}$: gelernte Parameter, Eingaben $\rightarrow$ Neuronen
- **Bias-Vektor** $\mathbf{b}$: gelernte Verschiebung
- **Gewichtete Summe**: $z = \mathbf{W}^T \mathbf{x} + \mathbf{b}$
- **Aktivierungsfunktion** $\sigma(\cdot)$: nicht-lineare Transformation auf z

Beispiel: ein Neuron mit 3 Eingaben:

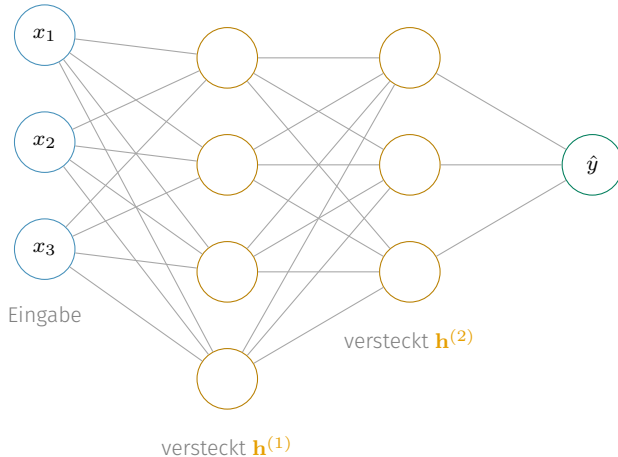


Vom Neuron zum Netz



1 Schicht: $\mathbf{h} = \sigma(W^T \mathbf{x} + \mathbf{b})$. Jede Zeile von W ist der Gewichtsvektor eines Neurons.

Vom Neuron zum Netz



Tief: $\mathbf{h}^{(1)} \rightarrow \mathbf{h}^{(2)} \rightarrow \hat{y}$. Jede Schicht speist die nächste; Tiefe erlaubt es, komplexe Merkmale Schritt für Schritt aufzubauen.

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Mathematische Tatsache: Die Komposition linearer Funktionen ist linear

$$\text{Schicht 2}(\text{Schicht 1}(\mathbf{x})) = \mathbf{W}_2^T (\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\text{immer noch linear in } \mathbf{x})$$

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Mathematische Tatsache: Die Komposition linearer Funktionen ist linear

$$\text{Schicht 2}(\text{Schicht 1}(\mathbf{x})) = \mathbf{W}_2^T(\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\text{immer noch linear in } \mathbf{x})$$

Lösung: Nach jeder Schicht eine **nicht-lineare Aktivierungsfunktion** $\sigma(\cdot)$ einfügen:

$$\mathbf{h} = \sigma(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

Nun ist: $\sigma(\text{Schicht 1}(\text{Schicht 2}(\mathbf{x})))$ **nicht-linear**!

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Mathematische Tatsache: Die Komposition linearer Funktionen ist linear

$$\text{Schicht 2}(\text{Schicht 1}(\mathbf{x})) = \mathbf{W}_2^T (\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\text{immer noch linear in } \mathbf{x})$$

Lösung: Nach jeder Schicht eine **nicht-lineare Aktivierungsfunktion** $\sigma(\cdot)$ einfügen:

$$\mathbf{h} = \sigma(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

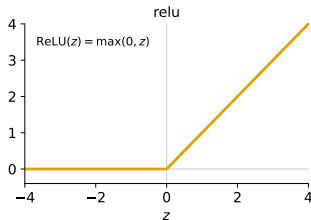
Nun ist: $\sigma(\text{Schicht 1}(\text{Schicht 2}(\mathbf{x})))$ **nicht-linear**!

Mächtigkeit: Mit genügend Schichten und Aktivierungsfunktionen lässt sich **jede** nicht-lineare Funktion lernen!

Häufige Aktivierungsfunktionen

ReLU:

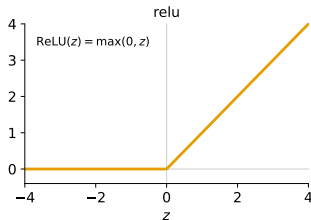
- einfach und schnell
- Standardwahl für versteckte Schichten
- Problem: „tote Neuronen“



Häufige Aktivierungsfunktionen

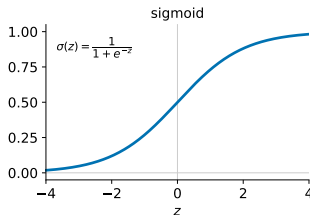
ReLU:

- einfach und schnell
- Standardwahl für versteckte Schichten
- Problem: „tote Neuronen“



Sigmoid:

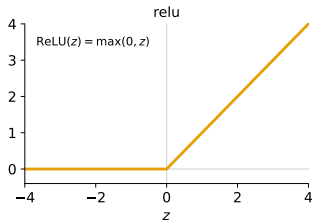
- Ausgaben zwischen 0 und 1
- gut für Ausgabeschichten
- langsames Training



Häufige Aktivierungsfunktionen

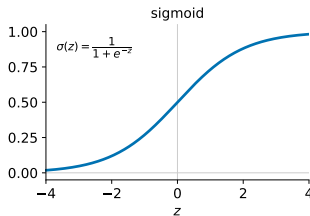
ReLU:

- einfach und schnell
- Standardwahl für versteckte Schichten
- Problem: „tote Neuronen“



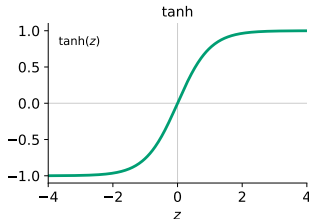
Sigmoid:

- Ausgaben zwischen 0 und 1
- gut für Ausgabeschichten
- langsames Training



Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

- Ausgaben zwischen -1 und 1
- stärkere Update-Signale
- gut für versteckte Schichten



Zusammenfassung

Ein neuronales Netz stapelt viele
Linear-und-Aktivierungs-Einheiten.

Jedes *Neuron*: $a = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$.

Die **Nichtlinearität** σ ist es, die **Tiefe** sinnvoll macht.

Mehr Schichten = mehr Kapazität = höheres Overfitting-Risiko
(wie wir in der Trainings-Lektion sehen werden).

Zusammenfassung

Ein neuronales Netz stapelt viele
Linear-und-Aktivierungs-Einheiten.

Jedes *Neuron*: $a = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$.

Die **Nichtlinearität** σ ist es, die **Tiefe** sinnvoll macht.

Mehr Schichten = mehr Kapazität = höheres Overfitting-Risiko
(wie wir in der Trainings-Lektion sehen werden).

- Logistische Regression = kleinstes Neuronales Netz mit Sigmoid-Ausgabe.
- Lineare Regression = kleinstes Neuronales Netz ohne Ausgabe-Aktivierung.

4. Theoretische Übungen

Übung 1: Der Kollaps

Eine Studentin zeigt ein „tiefes“ Netz: 5 gestapelte lineare Abbildungen $\mathbf{h}^{(\ell)} = W_{\ell}^T \mathbf{h}^{(\ell-1)} + \mathbf{b}_{\ell}$, mit $\mathbf{h}^{(0)} = \mathbf{x}$. Aktivierungsfunktionen wurden vergessen.

- (a) Man schreibe die Ausgabe des Netzes als eine einzige Formel in $\mathbf{x}$.
- (b) Man zeige, dass das Netz nicht ausdrucksstärker ist als eine *einzelne* lineare Einheit. Wie lauten die äquivalente Gewichtsmatrix und der äquivalente Bias?

Übung 1: Der Kollaps

Eine Studentin zeigt ein „tiefes“ Netz: 5 gestapelte lineare Abbildungen $\mathbf{h}^{(\ell)} = W_\ell^T \mathbf{h}^{(\ell-1)} + \mathbf{b}_\ell$, mit $\mathbf{h}^{(0)} = \mathbf{x}$. Aktivierungsfunktionen wurden vergessen.

- (a) Man schreibe die Ausgabe des Netzes als eine einzige Formel in $\mathbf{x}$.
- (b) Man zeige, dass das Netz nicht ausdrucksstärker ist als eine *einzelne* lineare Einheit. Wie lauten die äquivalente Gewichtsmatrix und der äquivalente Bias?

❗ (a) Schichtweises Einsetzen ergibt:
$$\hat{y} = W_5^T (W_4^T (W_3^T (W_2^T (W_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) + \mathbf{b}_4) + \mathbf{b}_5.$$

- (b) Ausmultiplizieren liefert $\hat{y} = W' \mathbf{x} + \mathbf{b}'$ mit $W' = W_5^T W_4^T W_3^T W_2^T W_1^T$ und $\mathbf{b}' = W_5^T W_4^T W_3^T W_2^T \mathbf{b}_1 + \dots + W_5^T \mathbf{b}_4 + \mathbf{b}_5$. Die Komposition linearer Abbildungen ist linear, also berechnet eine *einzelne* Schicht mit $(W', \mathbf{b}')$ dieselbe Funktion.

Übung 2: Forward Pass berechnen

Gegeben ein zweischichtiges Netz mit:

- Eingabe: $\mathbf{x} = [1, 2]^T$
- Gewichte Schicht 1: $\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, Bias: $\mathbf{b}_1 = [0.1, 0]^T$
- Aktivierung Schicht 1: ReLU
- Gewichte Schicht 2: $\mathbf{W}_2 = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}$, Bias: $b_2 = 0.2$
- Aktivierung Schicht 2: keine (lineare Ausgabe)

Übung 2: Forward Pass berechnen

Gegeben ein zweischichtiges Netz mit:

- Eingabe: $\mathbf{x} = [1, 2]^T$
- Gewichte Schicht 1: $\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, Bias: $\mathbf{b}_1 = [0.1, 0]^T$
- Aktivierung Schicht 1: ReLU
- Gewichte Schicht 2: $\mathbf{W}_2 = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}$, Bias: $b_2 = 0.2$
- Aktivierung Schicht 2: keine (lineare Ausgabe)

Man berechne den Vorwärtsthroughlauf: $\mathbf{z}_1 \rightarrow \mathbf{h}_1 \rightarrow z_2 \rightarrow \hat{y}$

Übung 2: Forward Pass berechnen

Gegeben ein zweischichtiges Netz mit:

- Eingabe: $\mathbf{x} = [1, 2]^T$
- Gewichte Schicht 1: $\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, Bias: $\mathbf{b}_1 = [0.1, 0]^T$
- Aktivierung Schicht 1: ReLU
- Gewichte Schicht 2: $\mathbf{W}_2 = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}$, Bias: $b_2 = 0.2$
- Aktivierung Schicht 2: keine (lineare Ausgabe)

Man berechne den Vorwärtsthroughlauf: $\mathbf{z}_1 \rightarrow \mathbf{h}_1 \rightarrow z_2 \rightarrow \hat{y}$

$$\textcircled{!} \quad \mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 0.4 \end{bmatrix}$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{z}_1) = \begin{bmatrix} 1.2 \\ 0.4 \end{bmatrix} \text{ (beide positiv, also unverändert)}$$

$$z_2 = \mathbf{W}_2^T \mathbf{h}_1 + b_2 = [0.4, 0.6] \begin{bmatrix} 1.2 \\ 0.4 \end{bmatrix} + 0.2 = 0.48 + 0.24 + 0.2 = 0.92$$

5. Programmierübung

Programmierung

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

<https://expert.ethz.ch/enrolled/SS26/mavt2/exercises>

■ Spirals

6. Training

Was ist Training?

Ein Modell zu **trainieren** bedeutet: jene Gewichte $\mathbf{w}$ wählen, die einen Verlust $\mathcal{L}(\mathbf{w})$ auf den *Trainingsdaten minimieren*.

Was ist Training?

Ein Modell zu **trainieren** bedeutet: jene Gewichte $\mathbf{w}$ wählen, die einen Verlust $\mathcal{L}(\mathbf{w})$ auf den *Trainingsdaten minimieren*.

Doch ein Modell, das auf den Trainingsdaten gut abschneidet, kann trotzdem unbrauchbar sein.

- Entscheidend ist, wie es sich auf Daten verhält, die es *nicht* gesehen hat.

Was ist Training?

Ein Modell zu **trainieren** bedeutet: jene Gewichte $\mathbf{w}$ wählen, die einen Verlust $\mathcal{L}(\mathbf{w})$ auf den *Trainingsdaten minimieren*.

Doch ein Modell, das auf den Trainingsdaten gut abschneidet, kann trotzdem unbrauchbar sein.

- Entscheidend ist, wie es sich auf Daten verhält, die es *nicht* gesehen hat.

Geringer Trainingsverlust $\neq$ gutes Modell.

Trainings- und Validierungsdaten

Man teilt die Daten in zwei Teile auf.

Trainings- und Validierungsdaten

Man teilt die Daten in zwei Teile auf.

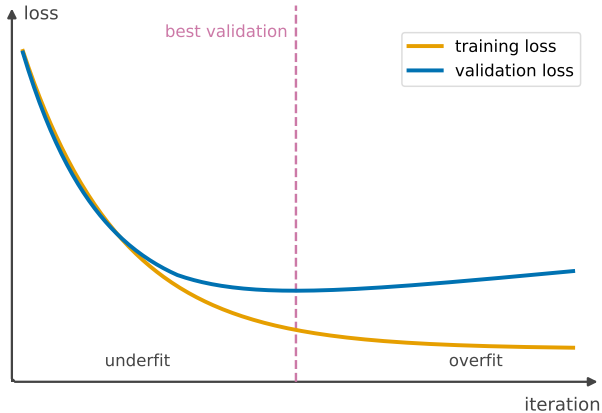
- **Trainingsmenge**: das Modell *sieht* diese Daten und passt seine Gewichte daran an.

Trainings- und Validierungsdaten

Man teilt die Daten in zwei Teile auf.

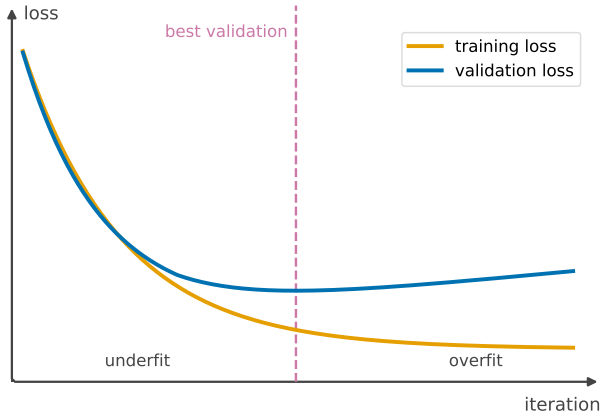
- **Trainingsmenge:** das Modell *sieht* diese Daten und passt seine Gewichte daran an.
- **Validierungsmenge:** das Modell *sieht* diese Daten während des Trainings *nie*. Sie dient dazu, zu prüfen, wie gut das Modell generalisiert.

Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

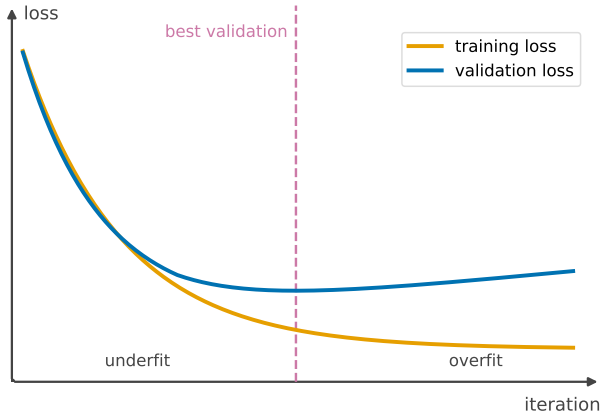
Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

- Anfangs: beide fallen gemeinsam.

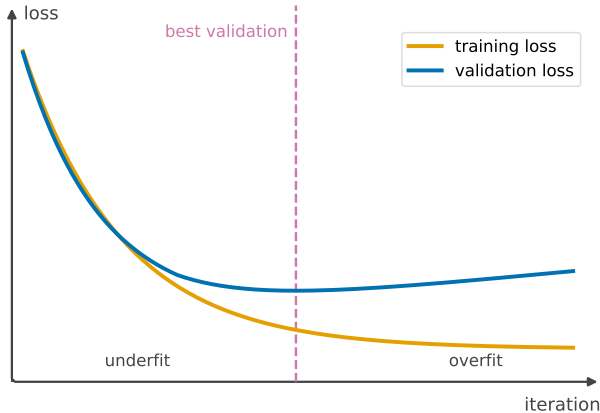
Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

- Anfangs: beide fallen gemeinsam.
- Später: **Trainingsverlust** fällt weiter, **Validierungsverlust** fällt nicht mehr und *steigt*.

Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

- Anfangs: beide fallen gemeinsam.
- Später: **Trainingsverlust** fällt weiter, **Validierungsverlust** fällt nicht mehr und *steigt*.
- Die **Lücke** ist das Signal.

Unteranpassung, gut, Überanpassung



Gleiche Daten, drei Modellkomplexitäten.

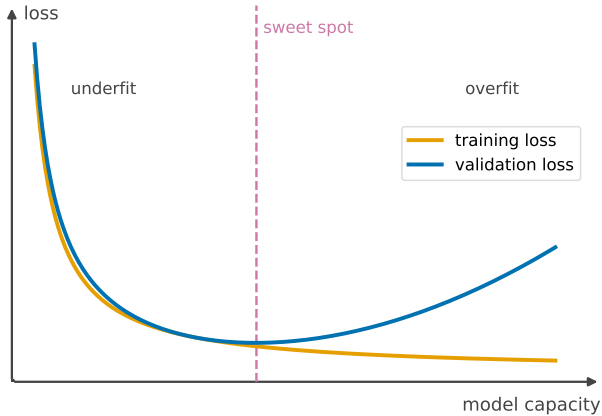
Links: zu einfach, die Gerade folgt dem Trend nicht (*Unteranpassung*: hoher Verlust überall).

Mitte: passend, erfasst das Muster und ignoriert das Rauschen.

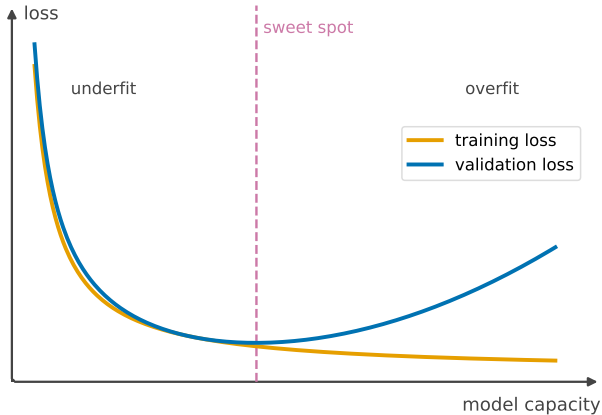
Rechts: zu flexibel, jagt jedem Punkt nach (*Überanpassung*: tiefer Trainings, hoher Validierungsverlust).

Modellkapazität

Kapazität = wie flexibel ein Modell ist.



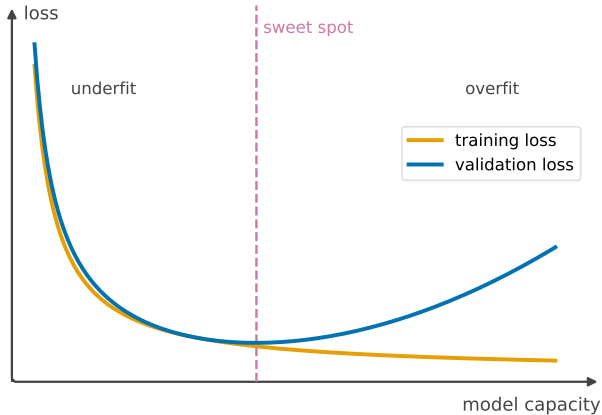
Modellkapazität



Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.

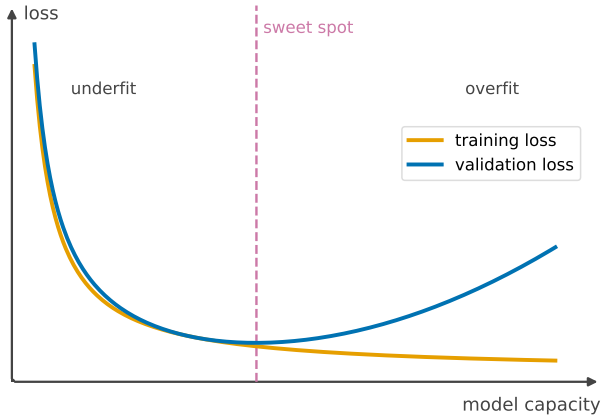
Modellkapazität



Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.
- Hohe Kapazität $\Rightarrow$ Überanpassung.

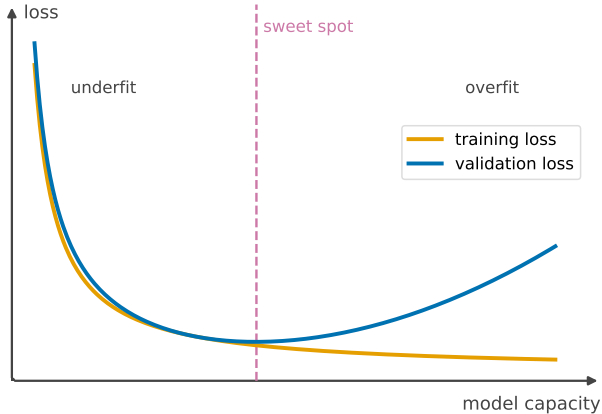
Modellkapazität



Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.
- Hohe Kapazität $\Rightarrow$ Überanpassung.
- Optimum dazwischen.

Modellkapazität

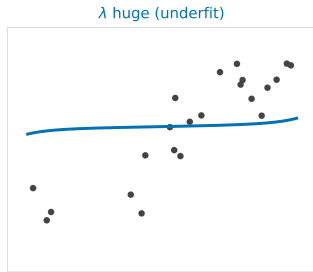
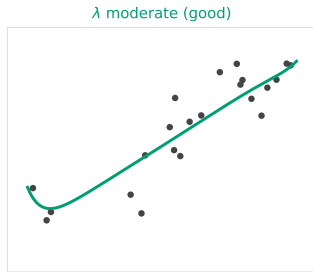
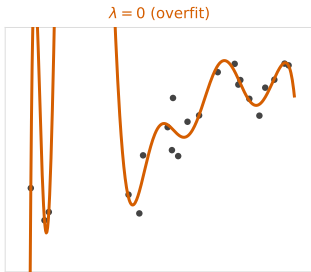


Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.
- Hohe Kapazität $\Rightarrow$ Überanpassung.
- Optimum dazwischen.

Stellschrauben für die Kapazität: *Grad* eines Polynoms, k bei k -NN, *Tiefe* eines Baumes, *Schichten* eines Netzes.

Regularisierung: ein Bild



Gleiche Daten, gleiche Modellfamilie (Polynom vom Grad 12), drei Werte der **Regularisierungsstärke** λ . Der Hebel λ bestimmt, wie stark grosse Gewichte bestraft werden.

Regularisierung: die Formel

Zum Verlust wird ein Strafterm für grosse Gewichte addiert:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \underbrace{\mathcal{L}(\mathbf{w})}_{\text{Datenanpassung}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{Strafterm}}$$

- Grosse Gewichte $\Rightarrow$ hoher Strafterm $\Rightarrow$ der Optimierer bevorzugt *kleine* Gewichte.
- Kleine Gewichte $\Rightarrow$ weniger zappeliges Modell $\Rightarrow$ weniger Überanpassung.

Regularisierung: die Formel

Zum Verlust wird ein Strafterm für grosse Gewichte addiert:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \underbrace{\mathcal{L}(\mathbf{w})}_{\text{Datenanpassung}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{Strafterm}}$$

- Grosse Gewichte $\Rightarrow$ hoher Strafterm $\Rightarrow$ der Optimierer bevorzugt *kleine* Gewichte.
- Kleine Gewichte $\Rightarrow$ weniger zappeliges Modell $\Rightarrow$ weniger Überanpassung.

Das ist L_2 -Regularisierung (auch Ridge genannt). Die Variante L_1 verwendet $\lambda \sum_d |w_d|$ und drückt manche Gewichte exakt auf null.

Parameter vs. Hyperparameter

Parameter	Hyperparameter
die Gewichte $\mathbf{w}$ <i>gelernt</i> während des Trainings	z. B. λ , Polynomgrad, k , Lernrate <i>gewählt</i> vor dem Training
bestimmt durch Verlust- minimierung auf der Trainings- menge	bestimmt auf der Validierungs- menge

Parameter vs. Hyperparameter

Parameter	Hyperparameter
die Gewichte $\mathbf{w}$ <i>gelernt</i> während des Trainings	z. B. λ , Polynomgrad, k , Lernrate <i>gewählt</i> vor dem Training
bestimmt durch Verlust- minimierung auf der Trainings- menge	bestimmt auf der Validierungs- menge

Parameter sind, was das Modell lernt. Hyperparameter sind die Stellschrauben, die wir vorgeben.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der Trainingsmenge trainieren, auf der Validierungsmenge auswerten.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der Trainingsmenge trainieren, auf der Validierungsmenge auswerten.
3. Den Kandidaten mit dem geringsten Validierungsverlust behalten.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der Trainingsmenge trainieren, auf der Validierungsmenge auswerten.
3. Den Kandidaten mit dem geringsten Validierungsverlust behalten.
4. Den endgültigen Verlust auf der Testmenge berichten, und zwar ein einziges Mal.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der **Trainings**menge trainieren, auf der **Validierung**smenge auswerten.
3. Den Kandidaten mit dem geringsten **Validierung**sverlust behalten.
4. Den endgültigen Verlust auf der **Test**menge berichten, und zwar ein einziges Mal.

Ist die Validierungsmenge klein und verrauscht, kann der gewählte Hyperparameter unglücklich ausfallen. Die Kreuzvalidierung (wie wir gleich sehen werden) schafft Abhilfe.

Zusammenfassung

Den Validierungsverlust im Auge behalten, nicht nur den Trainingsverlust.

Regularisierung und passende Kapazität bringen einen ins Optimum.

Hyperparameter auf der Validierungs, nicht der Trainingsmenge bestimmen.

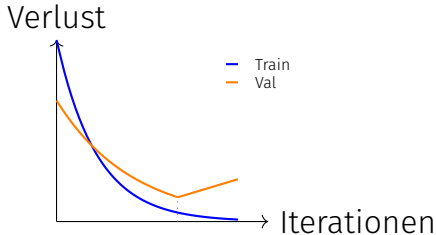
7. Theoretische Übungen

Übung 1: Verlustkurven lesen

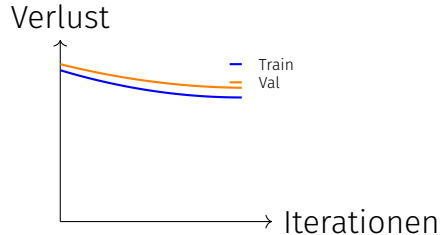
Für jedes Szenario sind Trainings- und Validierungsverlust gegen die Iterationszahl aufgetragen.

- (a) Trainingsverlust geht fast auf null; Validierungsverlust fällt zuerst, *steigt* dann und endet weit über dem Trainingsverlust.
- (b) Beide stagnieren früh auf einem hohen Wert.

Szenario (a):

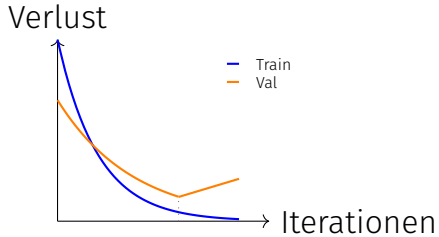


Szenario (b):

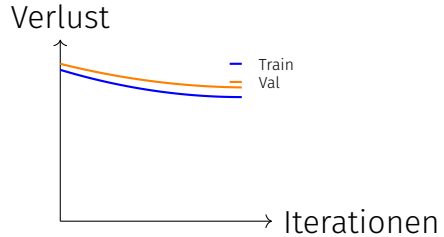


Übung 1: Verlustkurven lesen

Szenario (a):



Szenario (b):

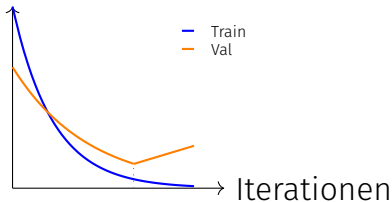


Für jedes: **Diagnose** und **empfohlene Massnahme**.

Übung 1: Verlustkurven lesen

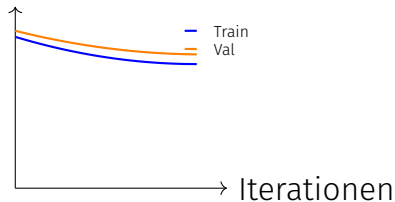
Szenario (a):

Verlust



Szenario (b):

Verlust



Für jedes: **Diagnose** und **empfohlene Massnahme**.

- ❗ (a) **Überanpassung**. Massnahmen: Early Stopping, λ erhöhen, Kapazität senken oder mehr Daten.
- (b) **Unteranpassung**. Massnahmen: flexibleres Modell, λ senken, mehr Epochen oder Merkmale.

Übung 2: Parameter oder Hyperparameter?

Man ordne jeden Punkt als **Parameter** (gelernt durch Verlustminimierung auf der Trainingsmenge) oder **Hyperparameter** (vor dem Training gewählt, auf der Validierungsmenge bestimmt) ein.

- (a) Das Gewicht w_3 in der linearen Regression.
- (b) Die Regularisierungsstärke λ .
- (c) Der Bias w_0 .
- (d) Die Lernrate des Optimierers.
- (e) Die Anzahl k der Nachbarn bei k -NN.
- (f) Die Anzahl Schichten eines neuronalen Netzes.

Übung 2: Parameter oder Hyperparameter?

Man ordne jeden Punkt als **Parameter** (gelernt durch Verlustminimierung auf der Trainingsmenge) oder **Hyperparameter** (vor dem Training gewählt, auf der Validierungsmenge bestimmt) ein.

- (a) Das Gewicht w_3 in der linearen Regression.
- (b) Die Regularisierungsstärke λ .
- (c) Der Bias w_0 .
- (d) Die Lernrate des Optimierers.
- (e) Die Anzahl k der Nachbarn bei k -NN.
- (f) Die Anzahl Schichten eines neuronalen Netzes.

❗ **Parameter:** (a) w_3 , (c) w_0 .

Hyperparameter: (b) λ , (d) Lernrate, (e) k , (f) Anzahl Schichten.

Die Unterscheidung gilt *allgemein*: jedes Modell hat beide Arten.

8. Programmierübung

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

<https://expert.ethz.ch/enrolled/SS26/mavt2/exercises>

- **Diabetes with regression trees and grid search**
- **Sine Regression**

9. Cross-Validation

Warum brauchen wir Validierung?

Ziel: Wir wollen abschätzen, wie gut ein Modell auf ungesehenen Daten abschneidet.

- Die Trainings-Performance ist meist zu optimistisch.
- Ein Modell kann die Trainingsdaten auswendig lernen.
- Deshalb brauchen wir Daten, die nicht für das Anpassen des Modells verwendet wurden.

Validierung hilft uns, die Generalisierungs-Performance abzuschätzen.

Das Problem mit einem einzelnen Split

Wenn wir die Daten nur einmal aufteilen, kann das Ergebnis stark vom Zufall abhängen.



Gleiches Modell, gleiche Daten, anderer Split, andere Schätzung.

k-Fold Cross-Validation

Idee: Verwende jeden Teil der Daten einmal zur Validierung.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Lauf 1	val	train	train	train	train	88%
Lauf 2	train	val	train	train	train	91%
Lauf 3	train	train	val	train	train	86%
Lauf 4	train	train	train	val	train	90%
Lauf 5	train	train	train	train	val	89%

- Teile die Daten in k Folds auf.
- Trainiere auf $k - 1$ Folds.
- Validiere auf dem verbleibenden Fold.
- Wiederhole, bis jeder Fold einmal zur Validierung verwendet wurde.

Wie werten wir das Ergebnis aus?

Nach der k -fold Cross-Validation erhalten wir k Validierungs-Scores:

$$s_1, s_2, \dots, s_k$$

Üblicherweise berichten wir den mittleren Validierungs-Score:

$$\bar{s} = \frac{1}{k} \sum_{i=1}^k s_i$$

- Der Mittelwert schätzt die erwartete Performance.
- Die Streuung zwischen den Scores zeigt, wie stabil das Modell ist.

Hyperparameter wählen

Cross-Validation wird häufig zur Wahl von Hyperparametern verwendet.

Beispiel für kNN:

$$k_{\text{kNN}} \in \{1, 3, 5, 7, 9\}$$

- Probiere mehrere Hyperparameter-Werte aus.
- Berechne für jeden Wert den Cross-Validation-Score.
- Wähle den Wert mit der besten mittleren Validierungs-Performance.

Wichtig: Die finale Testmenge sollte erst ganz am Ende verwendet werden.

Praktische Details

- Typische Wahl: $k = 5$ oder $k = 10$.
- Trainings- und Validierungs-Folds sind stets disjunkt.
- Der Split erfolgt üblicherweise zufällig, aber ohne Zurücklegen.

Spezialfall: Leave-One-Out Cross-Validation

- $k = n$, wobei n die Anzahl der Datenpunkte ist.
- Jeder Datenpunkt wird einmal als Validierungsmenge verwendet.
- Kann bei grossen Datensätzen rechenintensiv sein.

10. Theoretische Übungen

Aufgabe: k für kNN wählen

Wir führen eine 5-fold Cross-Validation auf einem kNN-Classifer für drei Kandidatenwerte von k durch. Die Validierungs-Genauigkeiten pro Fold lauten:

k_{kNN}	s_1	s_2	s_3	s_4	s_5
1	78%	82%	75%	80%	80%
3	86%	88%	84%	87%	85%
5	84%	85%	83%	86%	82%

Aufgaben:

- Berechne die mittlere Validierungs-Genauigkeit $\bar{s}$ für jeden Kandidaten.
- Welchen Wert von k_{kNN} sollten wir wählen?
- Wie viele Modell-Trainings hat die Cross-Validation insgesamt durchgeführt?

Lösung: k für kNN wählen

Mittlere Validierungs-Genauigkeit pro Kandidat:

$$\bar{s}_{k=1} = \frac{78+82+75+80+80}{5}\% = 79\%$$

$$\bar{s}_{k=3} = \frac{86+88+84+87+85}{5}\% = 86\%$$

$$\bar{s}_{k=5} = \frac{84+85+83+86+82}{5}\% = 84\%$$

Wähle $k_{\text{kNN}} = 3$, denn es hat die höchste mittlere Validierungs-Genauigkeit.

Gesamtanzahl Trainings während der CV: $G \cdot k = 3 \cdot 5 = 15$ Trainings.

Anschliessend trainiert man einmalig auf den vollen Trainingsdaten mit $k_{\text{kNN}} = 3$ und wertet dann genau einmal auf der Testmenge aus.

Vergleich der Modelle

Eigenschaft	Lin. Reg.	Log. Reg.	Entsch.-Bäume	k-NN
Aufgabe	Regression	Klassifikation	Beides	Beides
Entscheidungsgrenze	Linear	Linear	Achsenparallel	Nicht-linear
Interpretierbarkeit	Hoch	Hoch	Hoch	Mittel
Geschw. (Training)	Schnell	Schnell	Schnell	Sofort
Geschw. (Vorhersage)	Schnell	Schnell	Schnell	Langsam
Nicht-lineare Daten	Nein	Nein	Ja	Ja
Kategoriale Merkmale	Kodierung nötig	Kodierung nötig	Direkt	Kodierung nötig
Ausreisserempfindlichkeit	Hoch (MSE)	Niedrig	Hoch	Hoch
Haupt-Hyperparameter	–	–	max_depth	k
Overfitting-Risiko	Niedrig	Niedrig	Hoch	Hoch

Kurzleitfaden: Lineare Modelle sind schnell und interpretierbar, aber auf einfache Grenzen beschränkt. Bäume erfassen Nichtlinearität, lernen aber Rauschen auswendig. k-NN ist flexibel, aber langsam und braucht sorgfältige Vorverarbeitung.